```
clear all
clc
```

Funwork2

Phillipp Gery

Start with the non-linear model of the double inverted pendulum on a cart (DIPC) from FunWork 1.

```
syms theta1 theta2 x_pos theta1_dot theta2_dot x_dot
syms theta1_ddot theta2_ddot x_ddot
syms u x


%get Results from Funwork1 means Model
load("../Funwork1/Funwork1_Gery_Results.mat");
disp("non-linear model")
```

```
non-linear model
```

```
disp(f)
```

$$\left(\begin{array}{c} \dot{x} \\ \dot{\theta_1} \\ \dot{\theta_2} \\ -\frac{280\,u - 981\sin(2\,\theta_1) + 100\,\dot{\theta_1}^2\sin(\theta_1) + 45\,\dot{\theta_2}^2\sin(\theta_2) - 120\,u\cos(}{20\,\sigma_2} \\ \frac{1575\,\dot{\theta_2}^2\sin(\theta_1-\theta_2) - 30411\sin(\theta_1) - 8829\sin(\theta_1 - 2\,\theta_2) + 250\,\dot{\theta_1}^2\sin(2\,\theta_1) + 1400\,u\cos(\theta_1) + 450\,\dot{\theta_1}^2}{50\,\sigma_2} \\ -\frac{300\sin(\theta_1-\theta_2)\,\dot{\theta_1}^2 + 135\sin(\sigma_3)\,\dot{\theta_2}^2 - 2943\,\sigma_1 + 2943\sin(\theta_2) + 200\,u\cos(2}{75\cos(2\,\theta_1) + 135\cos(\sigma_3) - 390} \end{array}\right.$$

where

$$\sigma_1 = \sin(2\,\theta_1 - \theta_2)$$

$$\sigma_2 = 5\cos(2\,\theta_1) + 9\cos(\sigma_3) - 26$$

$$\sigma_3 = 2\,\theta_1 - 2\,\theta_2$$

```
%Jacobians and insert Numerical Values
% Define the equilibrium point (origin) again and Liarize model around Eq
% Point
```

```
eq_point_states = [x_pos, theta1, theta2, x_dot, theta1_dot, theta2_dot];
eq_point_values = [0, 0, 0, 0, 0, 0];
eq_input_value = 0;
%Jacobian parcial dervi
A_sym = jacobian(f, state_vector);
B_sym = jacobian(f, input_vector);
% variables and values  at the eq. point
all_sym_vars = [eq_point_states, u]; % states and input
all_num_vals = [eq_point_values, eq_input_value];
% Substitute all left values
A_at_eq = subs(A_sym, all_sym_vars, all_num_vals);
B_at_eq = subs(B_sym, all_sym_vars, all_num_vals);
% to numerical doubles
A = double(A_at_eq);
B = double(B_at_eq);
```

Problem 1.

Solve the Lyapunov matrix equation for the linearized open-loop system. Take .

Is the open-loop system asymptotically stable, that is, is the equilibrium state

of interest asymptotically stable in the sense of Lyapunov? Explain.

```
n = size(A,1);
Q = eye(n)
```

```
Q = 6×6
     1     0     0     0     0     0
     0     1     0     0     0     0
     0     0     1     0     0     0
     0     0     0     1     0     0
     0     0     0     0     1     0
     0     0     0     0     0     1
```

```
try
```

Solve Lyapunov fuction A'*P + P*A = -Q

```
P = lyap(A', Q);
```

if P is positive definite P is positive definite if and only if all its eigenvalues are positive.

```
eigenvalues_of_P = eig(P)
catch
   eig_A = eig(A)
   disp("Eigenvalues of A are NOT all on the open left Plain so the System
is NOT Asym Stable")
   disp("Becasue of this --> No Solution for Lyapunov")
end
```

```
eig_A = 6×1
```

```
         0
         0
   -9.1715
   -3.7394
    9.1715
    3.7394
Eigenvalues of A are NOT all on the open left Plain so the System is NOT Asym Stable
Becasue of this --> No Solution for Lyapunov
```

Problem 2.

(4 pts) Design a linear state-feedback controller for the linearized model.

Bad performance with convential Poleplacement

--> use Riccaty Controller

Define LQR weighting matrices Q and R

Q penalizes state errors.

10 for thettas to keep the angles small.

And 1 for x velocitys are not that important so 0.1

```
Q = diag([1, 10, 10, 0.1, 0.1, 0.1]);
```

R holds down contoller effort

```
R = 0.1 * eye(1);
% Calculate K using LQR
K = lqr(A, B, Q, R);
```

new System Matrix with Controller

```
A_cl = A-B*K
```

```
A_cl = 6×6
         0         0         0    1.0000         0         0
         0         0         0         0    1.0000         0
         0         0         0         0         0    1.0000
   -2.1082  181.5952 -220.2999   -4.6148   11.5266  -37.7577
    4.2164 -314.1405  411.1697    9.2296  -23.0532   75.5154
         0  -32.7000   32.7000         0         0         0
```

```
syscl = ss(A_cl,B,C,D);
Poles_cl = pole(syscl)
```

```
Poles_cl = 6×1 complex
  -9.2147 + 0.2435i
  -9.2147 - 0.2435i
  -3.8692 + 0.8501i
  -3.8692 - 0.8501i
  -0.7501 + 0.6721i
  -0.7501 - 0.6721i
```

All eigenvalues/Poles are on the open left Plane so the closed loop System is Stable

Problem 3.

(4 pts) Find the transfer function of the closed-loop system composed of the linearized model driven by the linear state-feedback controller.

```
syscl.InputName = 'u';
syscl.OutputName = {'x', 'theta1', 'theta2'};

G_cl = tf(syscl);
```

Transfer function from u to x:

```
G_cl(1,1) % first TF
```

```
ans =

  From input "u" to output "x":
     0.6667 s^4 - 5.625e-15 s^3 - 54.5 s^2 - 2.084e-13 s + 427.7
  --------------------------------------------------------------------
  s^6 + 27.67 s^5 + 283.5 s^4 + 1338 s^3 + 3001 s^2 + 2961 s + 1353

Continuous-time transfer function.
Model Properties
```

Transfer function from u to theta1:

```
G_cl(2,1) % second TF
```

```
ans =

  From input "u" to output "theta1":
   -1.333 s^4 + 2.216e-16 s^3 + 43.6 s^2 - 7.245e-15 s - 2.465e-28
  --------------------------------------------------------------------
  s^6 + 27.67 s^5 + 283.5 s^4 + 1338 s^3 + 3001 s^2 + 2961 s + 1353

Continuous-time transfer function.
Model Properties
```

Transfer function from u to theta2:

```
G_cl(3,1) %  third TF
```

```
ans =

  From input "u" to output "theta2":
                  43.6 s^2 - 4.745e-15 s + 1.21e-14
  --------------------------------------------------------------------
  s^6 + 27.67 s^5 + 283.5 s^4 + 1338 s^3 + 3001 s^2 + 2961 s + 1353

Continuous-time transfer function.
Model Properties
```

Problem 4.

(4 pts) Construct a Lyapunov function for the closed-loop system comprised of the linearized model driven by the state-feedback controller, solve the Lyapunov matrix equation, and check if the equilibrium state of interest of the closed-loop system is asymptotically stable in the sense of Lyapunov.

Theorem 4.1 (A. M. Lyapunov, 1892) The system $\dot{x} = Ax$, $x(t0) = x0$, is asymptoti- cally stable if and only if for any given real symmetric positive definite (r.s.p.d.) matrix

Q the solution P to the continuous Lyapunov matrix equation,AT P + P A = – Q (4.6) is also (real symmetric) positive definite.

```
try
```

Solve Lyapunov A'*P + P*A = -Q

```
P = lyap(A_cl', Q);

% P is positive definite if and only if all its eigenvalues are positive.
eigenvalues_of_P = eig(P)
catch
   eig_A = eig(A_cl)
   disp("Eigenvalues of Acl are not all on the open left Plain so the
System is not Asym Stable")
end
```

```
eigenvalues_of_P = 6×1
    0.0033
    0.0205
    0.2652
    0.7206
    5.2396
  192.0931
```

```
if all(eigenvalues_of_P > 0)
    disp('Conclusion: P is positive definite. The closed-loop system is
ASYMPTOTICALLY STABLE in the sense of Lyapunov.');
else
    disp('Conclusion: P is NOT positive definite. The closed-loop system is
NOT asymptotically stable in the sense of Lyapunov. ');
end
```

```
Conclusion: P is positive definite. The closed-loop system is ASYMPTOTICALLY STABLE in the sense of Lyapur
```

Problem 5.

(4 pts) Add an extra input in the double inverted pendulum on a cart (DIPC) non-linear model from the previous FunWork,namely u1 & u2 , the torque at the first joint. where is the force applied to the cart and is the torque applied at the first joint. In summary, we have now a three-output two-input system.

Design a stabilizing state-feedback controller using the linearized model. where is the force applied to the cart and is the torque applied at the first joint. In summary, we have now a three-output two-input system. Design a stabilizing state-feedback controller using the linearized model.

```
% add sys for the new input and params because again without subs
syms u1 u2
syms M m1 m2 l1 l2 g
%Values of parmas
M_val = 1.5;
m1_val = 0.5;
m2_val = 0.75;
l1_val = 0.5;
l2_val = 0.75;
g_val = 9.81;
%input vector update
input_vector = [u1; u2];
```

Because u2 is the tourqe on the first stick ist Q_2 of the Lagraian EoM

--> nessary to Update direct chnage in the nonlinear Model hard because of all the simplifications in the equation

```
% EOM --> x with u1
eq1 = (M + m1 + m2) * x_ddot ...
      + (m1 + m2) * l1 * theta1_ddot * cos(theta1) ...
      + m2 * l2 * theta2_ddot * cos(theta2) ...
      - (m1 + m2) * l1 * theta1_dot^2 * sin(theta1) ...
      - m2 * l2 * theta2_dot^2 * sin(theta2) ...
      - u1 == 0
```

eq1 = $-l_1 \sin(\theta_1)\ (m_1 + m_2)\, \dot{\theta_1}^2 - l_2\, m_2 \sin(\theta_2)\, \dot{\theta_2}^2 - u_1 + \ddot{x}\ (M + m_1 + m_2) + l_1\, \ddot{\theta_1} \cos(\theta_1)\ (m_1 + m_2) + l_2\, m_2\, \ddot{\theta_2}\, \mathrm{co}$

```
% EOM --> theta1 with added u2
eq2 = (m1 + m2) * l1^2 * theta1_ddot...
      + (m1 + m2) * l1 * x_ddot * cos(theta1) ...
      + m2 * l1 * l2 * theta2_ddot * cos(theta1 - theta2) ...
      + m2 * l1 * l2 * theta2_dot^2 * sin(theta1 - theta2) ...
      - (m1 + m2) * g * l1 * sin(theta1)- u2 == 0
```

eq2 = $l_1^2\, \ddot{\theta_1}\ (m_1 + m_2) - u_2 - g\, l_1 \sin(\theta_1)\ (m_1 + m_2) + l_1\, \ddot{x} \cos(\theta_1)\ (m_1 + m_2) + l_1\, l_2\, m_2\, \ddot{\theta_2} \cos(\theta_1 - \theta_2) + l_1\, l_2\, m_2\, \dot{\theta_2}$

```
% EOM --> theta2
eq3 = m2 * l2^2 * theta2_ddot ...
```

```
      + m2 * l2 * x_ddot * cos(theta2) ...
      + m2 * l1 * l2 * theta1_ddot * cos(theta1 - theta2) ...
      - m2 * l1 * l2 * theta1_dot^2 * sin(theta1 - theta2) ...
      - m2 * g * l2 * sin(theta2) == 0
```

eq3 = $l_2^2\, m_2\, \dot{\theta_2} - g\, l_2\, m_2 \sin(\theta_2) + l_2\, m_2\, \ddot{x} \cos(\theta_2) + l_1\, l_2\, m_2\, \dot{\theta_1} \cos(\theta_1 - \theta_2) - l_1\, l_2\, m_2\, \dot{\theta_1}^2 \sin(\theta_1 - \theta_2) = 0$

```
%perform again Linarization as done in Funwork1
% copy code
%Solve equations for higthes derviative
eqddot = solve([eq1, eq2, eq3], [x_ddot, theta1_ddot, theta2_ddot]);
% nonlinear System Model f(x,u)
f_u2 = [x_dot;
     theta1_dot;
     theta2_dot;
     eqddot.x_ddot
     eqddot.theta1_ddot;
     eqddot.theta2_ddot;];

%sub into f
f_u2 = simplify(subs(f_u2, {M, m1, m2, l1, l2, g}, {M_val, m1_val, m2_val,
l1_val, l2_val, g_val}));

%Jacobians and insert Numerical Values
% Define the equilibrium point (origin) again and Liarize model around Eq
% Point
eq_point_states = [x_pos, theta1, theta2, x_dot, theta1_dot, theta2_dot];
eq_point_values = [0, 0, 0, 0, 0, 0];
%new Input Vector with 2 time = 0 --> Optimal case no force from outside
eq_input_value = [0 0];
%Jacobian parcial dervi
A_sym = jacobian(f_u2, state_vector);
B_sym = jacobian(f_u2, input_vector);
% variables and values  at the eq. point
all_sym_vars = [eq_point_states, input_vector.']; % states and input
all_num_vals = [eq_point_values, eq_input_value];
% Substitute all left values
A_at_eq = subs(A_sym, all_sym_vars, all_num_vals);
B_at_eq = subs(B_sym, all_sym_vars, all_num_vals);
% to numerical doubles
A_u2 = double(A_at_eq);
B_u2 = double(B_at_eq);
```

new Jacobian Matrix A (Linarized) for System with 2*u:

Stays the same because the System remains the same only additional Input

```
disp(A_u2); %
```

```
         0         0         0    1.0000         0         0
         0         0         0         0    1.0000         0
         0         0         0         0         0    1.0000
         0   -8.1750         0         0         0         0
         0   65.4000  -29.4300         0         0         0
         0  -32.7000   32.7000         0         0         0
```

new Jacobian Matrix B (Linarized) for System with 2*u:

```
disp(B_u2);
```

```
         0         0
         0         0
         0         0
    0.6667   -1.3333
   -1.3333   10.6667
         0   -5.3333
```

Matrix C for System with 2*u stay s the same:

```
C_u2 = C;
disp(C_u2)
```

```
     1     0     0     0     0     0
     0     1     0     0     0     0
     0     0     1     0     0     0
```

new Matrix D for System with 2*u:

```
D_u2 = zeros(3, 2);
disp(D_u2)
```

```
     0     0
     0     0
     0     0
```

Controller Design Again, for two input states

--> use Riccaty Controller AGAIN for new Controller

```
% Define LQR weighting matrices Q and R
% Q penalizes state errors. We want to keep the angles small.
Q_u2 = diag([1, 10, 10, 0.1, 0.1, 0.1]);
% R penalizes control effort.
R_u2 = 0.1 * eye(2); % thsi time eye(2) because we have 2 inputs
% Calculate K using LQR
K_u2 = lqr(A_u2, B_u2, Q_u2, R_u2);


% NEW closed-loop system matrix
A_clu2 = A_u2 - B_u2*K_u2;
```

Find the poles (eigenvalues) of the closed-loop system to verify stability

Poles of the System with 2 Inputs and new Controller

```
poles_Aclu2 = eig(A_clu2)
```

```
poles_Aclu2 = 6×1 complex
 -14.5592 + 0.0000i
 -10.0674 + 0.0000i
  -3.9484 + 1.0654i
  -3.9484 - 1.0654i
  -0.7584 + 0.6931i
  -0.7584 - 0.6931i
```

```
% Check stability
if all(real(poles_Aclu2) < 0)
    disp('Conclusion: All poles have negative real parts. The closed-loop
system is ASYMPTOTICALLY STABLE.');
else
    disp('Conclusion: At least one pole has a non-negative real part. The
system is NOT stable.');
end
```

```
Conclusion: All poles have negative real parts. The closed-loop system is ASYMPTOTICALLY STABLE.
```

Problem 6.

(4 pts) Use the linearized model to design a Luenberger observer

define the Poles

Literature Rule of the Thump:

oberserver Poles *(2-6) = Controller Poles

```
p_obs = poles_Aclu2*2;
```

Calculate the observer matrix L

```
L = place(A', C', p_obs)'
```

```
L = 6×3
   49.2532         0         0
         0    9.4356    2.2394
         0   -0.0274    9.3918
  586.2936   -8.1750         0
         0   80.1529  -36.7675
         0  -25.5765   48.3036
```

Verify stability

```
A_obs = A - L*C;
observer_eig = eig(A_obs)
```

```
observer_eig = 6×1 complex
 -29.1183 + 0.0000i
 -20.1349 + 0.0000i
```

```
  -7.8969 + 2.1308i
  -7.8969 - 2.1308i
  -1.5168 + 1.3862i
  -1.5168 - 1.3862i
```

```
if max(real(observer_eig)) < 0
    disp('Conclusion: All observer poles are in the left-half plane. The
observer is stabe!!');
else
    disp('Conclusion: The observer is not stable.');
end
```

```
Conclusion: All observer poles are in the left-half plane. The observer is stabe!!
```

Extended StateSpace Model

```
A_ex = [A_u2 -B_u2*K_u2;
        L*C_u2 A_u2-B_u2*K_u2-L*C_u2]
```

```
A_ex = 12×12
         0         0         0    1.0000         0         0         0         0 ···
         0         0         0         0    1.0000         0         0         0
         0         0         0         0         0    1.0000         0         0
         0   -8.1750         0         0         0         0   -4.2367   15.8420
         0   65.4000  -29.4300         0         0         0   33.7705  -79.8180
         0  -32.7000   32.7000         0         0         0  -16.8647   32.0894
   49.2532         0         0         0         0         0  -49.2532         0
         0    9.4356    2.2394         0         0         0         0   -9.4356
         0   -0.0274    9.3918         0         0         0         0    0.0274
  586.2936   -8.1750         0         0         0         0 -590.5303   15.8420
      ⋮
```

```
B_ex = [B_u2;
        B_u2]
```

```
B_ex = 12×2
         0         0
         0         0
         0         0
    0.6667   -1.3333
   -1.3333   10.6667
         0   -5.3333
         0         0
         0         0
         0         0
    0.6667   -1.3333
      ⋮
```

```
C_ex = [C_u2 -D_u2*K_u2]
```

```
C_ex = 3×12
     1     0     0     0     0     0     0     0     0     0     0     0
     0     1     0     0     0     0     0     0     0     0     0     0
     0     0     1     0     0     0     0     0     0     0     0     0
```

```
D_ex = [D_u2]
```

```
D_ex = 3×2
     0     0
     0     0
     0     0
```

Problem 7.

(4 pts) Construct a Lyapunov function for the closed-loop system composed of the linearized model driven by the combined controller-observer compensator,solve the Lyapunov matrix equation, using and check if the equilibrium state of interest of the closed-loop systemis asymptotically stable in the sense of Lyapunov.

```
Q_ex = eye(12);

try
```

Solve Lyapunov A'*P + P*A = -Q

```
P_ex = lyap(A_ex', Q_ex);
```

P is positive definite if and only if all its eigenvalues are positive.

```
eig_of_P_ex = eig(P_ex)
catch
   eig_A_ex = eig(A_ex);
   disp(eig_A_ex)
   if all(eig_A_ex > 0)
        disp("Eigenvalues of A_ex are not all on the open left Plain so the
System is not Asym Stable")
   end
end
```

```
eig_of_P_ex = 12×1
    0.0068
    0.0331
    0.0543
    0.2430
    0.4941
    1.3298
    1.8587
    2.8706
   14.7795
   49.6403
      ⋮
```

```
if all(eig_of_P_ex > 0)
    disp('Conclusion: All eig(P) are possitive --> P_ex is positive
definite. The closed-loop system is ASYMPTOTICALLY STABLE in the sense of
Lyapunov.');
else
    disp('Conclusion: P_ex is NOT positive definite. The closed-loop system
is NOT asymptotically stable in the sense of Lyapunov. ');
end
```

```
Conclusion: All eig(P) are possitive --> P_ex is positive definite. The closed-loop system is ASYMPTOTICAL
```

Problem 8.

(4 pts) Find the transfer function of the closed-loop system composed of the linearized model, driven by the combined controller-observer compensator.

```
sys_obs = ss(A_ex, B_ex, C_ex , D_ex);
sys_obs.InputName = {'u1', 'u2'};
sys_obs.OutputName = {'x', 'theta1', 'theta2'};
```

transfer function G(s) = C*(sI-A)^-1*B + D

```
G_obs = tf(sys_obs);
%simplify transfer function
G_obssimp = minreal(G_obs);


% other way to calc tf of Sys without extendet model
sys4 = ss(A_u2-B_u2*K_u2,B_u2,C_u2, D_u2);
G_obs2 = minreal(tf(sys4));
```

Transfer function from u1 to x:

```
G_obssimp(1,1)
```

```
ans =

  From input "u1" to output "x":
        0.6667 s^4 + 21.11 s^3 + 155.1 s^2 + 268.9 s + 7.987
  --------------------------------------------------------------
  s^6 + 34.04 s^5 + 408.2 s^4 + 2146 s^3 + 5209 s^2 + 5375 s + 2588

Continuous-time transfer function.
Model Properties
```

Transfer function from u1 to theta1:

```
G_obssimp(2,1)
```

```
ans =

  From input "u1" to output "theta1":
        -1.333 s^4 - 55.01 s^3 - 294.5 s^2 - 370.5 s - 220.6
  --------------------------------------------------------------
  s^6 + 34.04 s^5 + 408.2 s^4 + 2146 s^3 + 5209 s^2 + 5375 s + 2588

Continuous-time transfer function.
Model Properties
```

Transfer function from u1 to theta2:

```
G_obssimp(3,1)
```

```
ans =

  From input "u1" to output "theta2":
           8.522 s^3 - 10.43 s^2 + 5.675e-13 s + 4.43e-13
  -------------------------------------------------------------------
  s^6 + 34.04 s^5 + 408.2 s^4 + 2146 s^3 + 5209 s^2 + 5375 s + 2588

Continuous-time transfer function.
Model Properties
```

Transfer function from u2 to x:

```
G_obssimp(1,2)
```

```
ans =

  From input "u2" to output "x":
        -1.333 s^4 - 8.488 s^3 - 30.37 s^2 - 325.3 s - 818.2
  -------------------------------------------------------------------
  s^6 + 34.04 s^5 + 408.2 s^4 + 2146 s^3 + 5209 s^2 + 5375 s + 2588

Continuous-time transfer function.
Model Properties
```

Transfer function from u2 to theta1:

```
G_obssimp(2,2)
```

```
ans =

  From input "u2" to output "theta1":
         10.67 s^4 + 61.51 s^3 + 81.38 s^2 + 68.01 s - 2.153
  -------------------------------------------------------------------
  s^6 + 34.04 s^5 + 408.2 s^4 + 2146 s^3 + 5209 s^2 + 5375 s + 2588

Continuous-time transfer function.
Model Properties
```

Transfer function from u2 to theta2:

```
G_obssimp(3,2)
```

```
ans =

  From input "u2" to output "theta2":
    -5.333 s^4 - 29.69 s^3 - 83.52 s^2 - 3.199e-13 s - 6.164e-14
  -------------------------------------------------------------------
  s^6 + 34.04 s^5 + 408.2 s^4 + 2146 s^3 + 5209 s^2 + 5375 s + 2588

Continuous-time transfer function.
Model Properties
```

Problem 9

Part 1. (4 pts) Implement and perform animation of the closed-loop system composed of the combined controller-observer compensator driving the DIPC nonlinear model.

```
%  physical parameters
l1 = 0.5;
```

```
l2 = 0.75;
g = 9.81;


% Initial Conditions
x_pos_init      = 0;
theta1_init     = 0.15;
theta2_init     = -0.2;
x_dot_init      = 0;
theta1_dot_init = 0;
theta2_dot_init = 0;

% Init  state vector for the loop
x_vec = [x_pos_init; theta1_init; theta2_init; x_dot_init; theta1_dot_init;
theta2_dot_init];


%Simtime
tfinal = 20;       % seconds


% Cart dimensions
cart_width = 0.5;
cart_height = 0.25;

% Simtime
tspan = [0 tfinal];

x_vec_init = [x_pos_init; theta1_init; theta2_init; x_dot_init;
theta1_dot_init; theta2_dot_init];
x_hat_init = zeros(6, 1);
```

Create extendet state Space vector with estimated and Simualted States

```
x_ex_int = [x_vec_init; x_hat_init];


% Desired Pos -> w Controller Input Add on to do fancy shit
x_desired = 0;
theta1_desired = 0;
theta2_desired = 0;
w = [x_desired; theta1_desired; theta2_desired; 0; 0; 0];
```

Function definion of Nonlinear Model

```
f_handle = matlabFunction(f_u2, 'Vars', {state_vector, input_vector});
```

The Obsever Model with the estimated State Vector can be derviates as folwing:

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - \hat{y})$$

$$\dot{\hat{\mathrm{x}}} = \mathrm{A}\hat{\mathrm{x}} + \mathrm{Bu} + \mathrm{L}(\mathrm{Cx} - \mathrm{C}\hat{\mathrm{x}})$$

$$\dot{\hat{x}} = A\hat{x} + Bu + LCx - LC\hat{x}$$

$$\dot{\hat{\mathrm{x}}} = (\mathrm{A} - \mathrm{LC})\hat{\mathrm{x}} + \mathrm{Bu} + \mathrm{LCx}$$

the Nonlinear Function stays the same f(u,x)

```
% Function Handle for the extendet Sys
combined_dynamics = @(t, x_ex) [f_handle(x_ex(1:6), -K_u2 * (x_ex(7:12) -
w)); ...  % NonLin Plant
    (A_u2 - L*C_u2)*x_ex(7:12) + B_u2*(-K_u2*(x_ex(7:12) - w)) +
L*C_u2*x_ex(1:6)]; % Observer

[T, X] = ode45(combined_dynamics, tspan, x_ex_int);

%Output for 3D Animation
x_res = timeseries(X(:,1), T);
Theta1_res = timeseries(X(:,2), T);
Theta2_res = timeseries(X(:,3), T);
```